

Greedy algorithm

①

Algorithm for optimization problem goes through a sequence of steps making with a set of choices at each step. For many optimization problem using dynamic programming to determine best choice is overkill. A Greedy Algorithm always makes the choice a that looks best at the moment. ~~that each~~ it makes locally optimal choice in the hope that this choice will lead to globally optimal solution.

Greedy always do not always yield optimal solutions but for many problems they do.

Example Scheduling several competing activities that require exclusive use of common resource.

Goal: selecting maximum sized set of mutually compatible activities

Suppose $S = \{a_1, a_2, \dots, a_n\}$ of n proposed activities wish to use resource.

Each activity a_i has a start time s_i and finish time f_i where $0 \leq s_i \leq f_i < \infty$

3

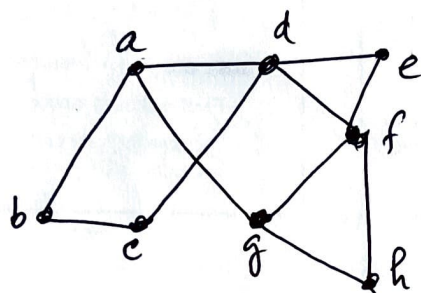
Greedy algorithm

①

1st example

Independent set: in a graph G is a subset U of the vertex set such that $v(u)$

st. for any $x, y \in U$ $(x, y) \notin E(G)$.



Example independent set

$\{a, c, f\}$

$\{a, c\}$ is also an i.s.

$\{b, f, \}$ $\{e, h, c, a\}$ There are all independent set
we have listed out 4 such Independent set but
there are many others.

Each vertex by itself is an Independent set. So formally Independent set is
a collection of vertices so that there are no
edges between them.

Example of Radio Station This one
is an application.

There are other places when one can look for
Independent set.

②

Well The general problem is; given a graph we want to find an Independent set of large size

This problem is turn out to be hard and look at it later in the course. For now we restrict The graph to be trees. So the problem now becomes.

input: A tree T

output: Maximum sized
Independent set in T

Tree is a graph which is connected and acyclic.

So I want a subset of the vertex set in T of maximum size such that there is no edge between any pair of vertices.

So how do we find maximum sized Independent set in T .

Suppose, T has n vertices

One way to do it is to sort of brute force approach, which looks for all possible subset of vertex set look at whether they are Indepe if they are Independent set and choose the

(3)

One which has maximum size. But this approach is too expensive, the time taken is atleast the ~~number~~ number of subsets of n which is 2^n . We want to do it much faster.

well what else one could do is pick a vertex, ~~to~~ and ~~once~~ in the independent set then the neighbors of it this will not be included in the Independent set. So that is by definition of the Independent Set.

So, pick a vertex

- ~~through~~ out throw out its neighbor
- look at the graph again pick a vertex ~~that~~ that remain throw out its neighbor and so on

This is how you get an independent set that is fine but how do you ensure ~~that~~ that its size is maximum.

If you pick a ~~wrong~~ wrong vertex at any point you will be in ~~trouble~~ trouble

So here is simple example

conditn this graph ④



If we pick vertex a then we get an independent set of size one whereas the optimum is of size three.

The solution to the problem is subset of the vertex set and you have to somehow pick the right-subset. and the greedy way says pick one by one somehow.

Now which vertex you would like to put in the Independent set so that it becomes large.

Intuition tells you that you should pick a vertex of smallest degree in this graph

If you pick a vertex of smallest degree then the number of vertex you throw up will be smaller. In particular if we pick a leaf node in the independent set then we throw out just only one vertex, and this actually works. in the sense that it actually produces maximum size Independent set.

⑤
We need a proof that this actually produces maximum size Independent set.

The algorithm is just this

- pick a leaf
- throw out its neighbors and continue.
- at each stage we will have forest

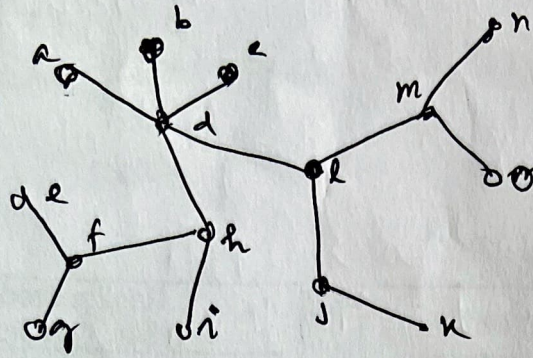
If we have a vertex of degree zero, it is not adjacent to any other vertex which we will come across as the algorithm proceeds. Put this vertex in the Independent set blindly.

If no such ~~vertex~~ vertex exist then we need to pick ^{any} a leaf, ~~for~~ (from ^{any of} the (many) trees as the algorithm proceeds) and put it in the Independent set and continue.

This algorithm works however it needs a proof.

The algorithm is greedy in the sense that we started with a piece and have a part of the solution and do not change it we add new vertices to it and never backtrack.

another way to look at the problem and come to the same algorithm.



This is the tree given

Find Max Independent set in this tree

Suppose we have some subset which is an Independent set

$$S = \{c, f, i, j, m\}$$

The algorithm has not picked it. ~~This is~~
 Suppose somebody has given the set. This
 is to get your thinking process going

Now the question, Can I add and subtract
 vertices to this solution to increase the
 size.

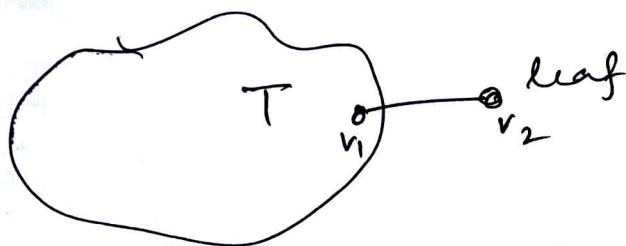
~~if~~ if you look carefully we can
 add n & o and remove m .

From the example actually it looks like
 we can always do this with the leaf.

(7)

so This naturally raise the question That, do all leaves always fall in some optimum Independent set,?

The answer is yes? why?
well supposing we have a tree and leaf as follows



well what can I say here. one of these ^{optimal} v_1 or v_2 better be in the Independent set
because if v_1 is not in the IS then we can add v_2 in IS.

So suppose v_1 is ~~not~~ in the IS then we can add v_2 to IS and the size of IS does not change, The set is ~~still~~ ^{still} IS and the size does not change

We picked one ~~element~~ ^{element} out and put one element in and the size does not change. This is the exchange trick that we did to increase the size of the IS.

⑧

So this is a one line proof that all leaves should be in ~~the~~ some optimum @ IS.

- So we can start of ~~it~~ by putting all leaves in the IS as there is an optimum IS that contains these leaves.

→ Remove the neighbor and

← Recurse on the remaining

This is good thing about greedy algorithm if you pick a part of the output then you have a smaller input to deal with and you can apply the same technique over and over again.

Summary of the algorithm

- ~~put~~ put all the leaves in the IS

- Remove the neighbors

- The tree will now have ^{many} ~~new~~ parts

- Recurse on each part.